LAB PROGRAME

CHAPTER 2 : BRUTE FORCE

NAME: S. BHUVANESH

REG NO: 192521004

1. Write a program to perform the following
   - An empty list
   - A list with one element
   - A list with all identical elements
   - A list with negative numbers

   Test Cases:
   1. **Input:** []
      - **Expected Output:** []
   2. **Input:** [1]
      - **Expected Output:** [1]
   3. **Input:** [7, 7, 7, 7]
      - **Expected Output:** [7, 7, 7, 7]
   4. **Input:** [-5, -1, -3, -2, -4]
      - **Expected Output:** [-5, -4, -3, -2, -1]

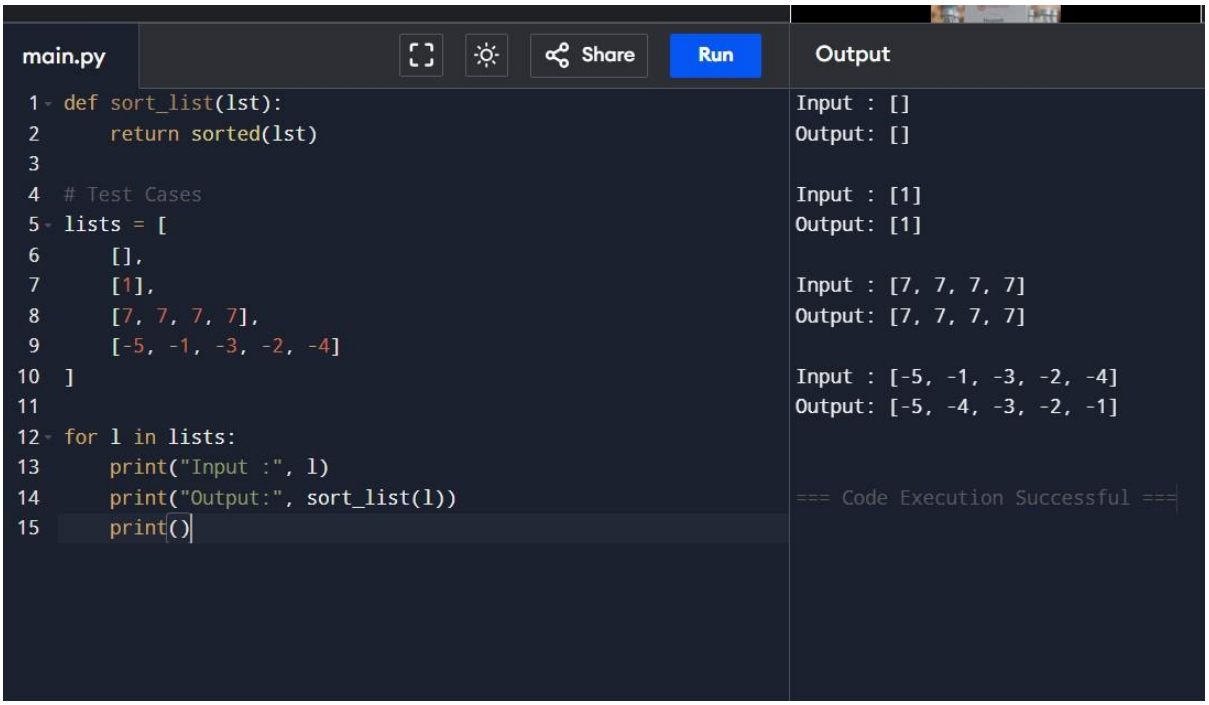
CODE:

```
def sort_list(lst):
    return
    sorted(lst)

lists = [
    [],

    [1],
    [7, 7, 7, 7],
    [-5, -1, -3, -2, -4]
]
for l in lists:
    print("Input :",
    l)
```

print("Output:",

sort_list(l)) print()

OUTPUT:



2.Describe the Selection Sort algorithm's process of sorting an array. Selection Sort works by dividing the array into a sorted and an unsorted region. Initially, the sorted region is empty, and the unsorted region contains all elements. The algorithm repeatedly selects the smallest element from the unsorted region and swaps it with the leftmost unsorted element, then moves the boundary of the sorted region one element to the right. Explain why Selection Sort is simple to understand and implement but is inefficient for large datasets. Provide examples to illustrate step-by-step how Selection Sort rearranges the elements into ascending order, ensuring clarity in your explanation of the algorithm's mechanics and effectiveness.

Sorting a Random Array:

Input: [5, 2, 9, 1, 5, 6]
Output: [1, 2, 5, 5, 6, 9]
Sorting a Reverse Sorted Array:
Input: [10, 8, 6, 4, 2]
Output: [2, 4, 6, 8, 10]
Sorting an Already Sorted Array:
Input: [1, 2, 3, 4, 5]
Output: [1, 2, 3, 4, 5]

CODE

```
def
    selection_sort(arr):
    n = len(arr)

    for i in range(n):
        min_index = i

        for j in range(i + 1, n):
            if arr[j] < arr[min_index]:
                min_index = j

        arr[i], arr[min_index] = arr[min_index], arr[i]
    return arr

print(selection_sort([5, 2, 9, 1, 5, 6]))
print(selection_sort([10, 8, 6, 4, 2]))
print(selection_sort([1, 2, 3, 4, 5]))
```

OUTPUT

main.py

Share Run

```python
def selection_sort(arr):
    n = len(arr)

    for i in range(n):
        min_index = i

        for j in range(i + 1, n):
            if arr[j] < arr[min_index]:
                min_index = j

        arr[i], arr[min_index] = arr[min_index], arr[i]

    return arr


print(selection_sort([5, 2, 9, 1, 5, 6]))
print(selection_sort([10, 8, 6, 4, 2]))
print(selection_sort([1, 2, 3, 4, 5]))
```

Output

```
[1, 2, 5, 5, 6, 9]
[2, 4, 6, 8, 10]
[1, 2, 3, 4, 5]

=== Code Execution Successful
```

3.Write code to modify bubble_sort function to stop early if the list becomes sorted before all passes are completed.

.

main.py | Share | Run | Output

```python
def bubble_sort(arr):
    n = len(arr)

    for i in range(n):
        swapped = False

        for j in range(0, n - i - 1):

            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swapped = True

        if not swapped:
            break

    return arr


print(bubble_sort([64,25,12,22,11]))
print(bubble_sort([29,10,14,37,13]))
print(bubble_sort([3,5,2,1,4]))
print(bubble_sort([1,2,3,4,5]))
print(bubble_sort([5,4,3,2,1]))
```

```
[11, 12, 22, 25, 64]
[10, 13, 14, 29, 37]
[1, 2, 3, 4, 5]
[1, 2, 3, 4, 5]
[1, 2, 3, 4, 5]

=== Code Execution Successful ===
```

4.Write code for Insertion Sort that manages arrays with duplicate elements during the sorting process. Ensure the algorithm's behavior when encountering duplicate values, including whether it preserves the relative order of duplicates and how it affects the overall sorting outcome

main.py | Share | Run | Output

```python
def insertion_sort(arr):

    for i in range(1, len(arr)):

        key = arr[i]
        j = i - 1

        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j -= 1

        arr[j + 1] = key

    return arr


print(insertion_sort([3,1,4,1,5,9,2,6,5,3]))
print(insertion_sort([5,5,5,5,5]))
print(insertion_sort([2,3,1,3,2,1,1,3]))
```

```
[1, 1, 2, 3, 3, 4, 5, 5, 6, 9]
[5, 5, 5, 5, 5]
[1, 1, 1, 2, 2, 3, 3, 3]

=== Code Execution Successful ===
```

5.Given an array arr of positive integers sorted in a strictly increasing order, and an

integer k. return the kth positive integer that is missing from this array.

Example 1:

Input: arr = [2,3,4,7,11], k = 5

Output: 9

Explanation: The missing positive integers are [1,5,6,8,9,10,12,13,...]. The 5th missing positive integer is 9.

Example 2:

Input: arr = [1,2,3,4], k =

2 Output: 6

Explanation: The missing positive integers are [5,6,7,...]. The 2nd missing positive integer is 6.

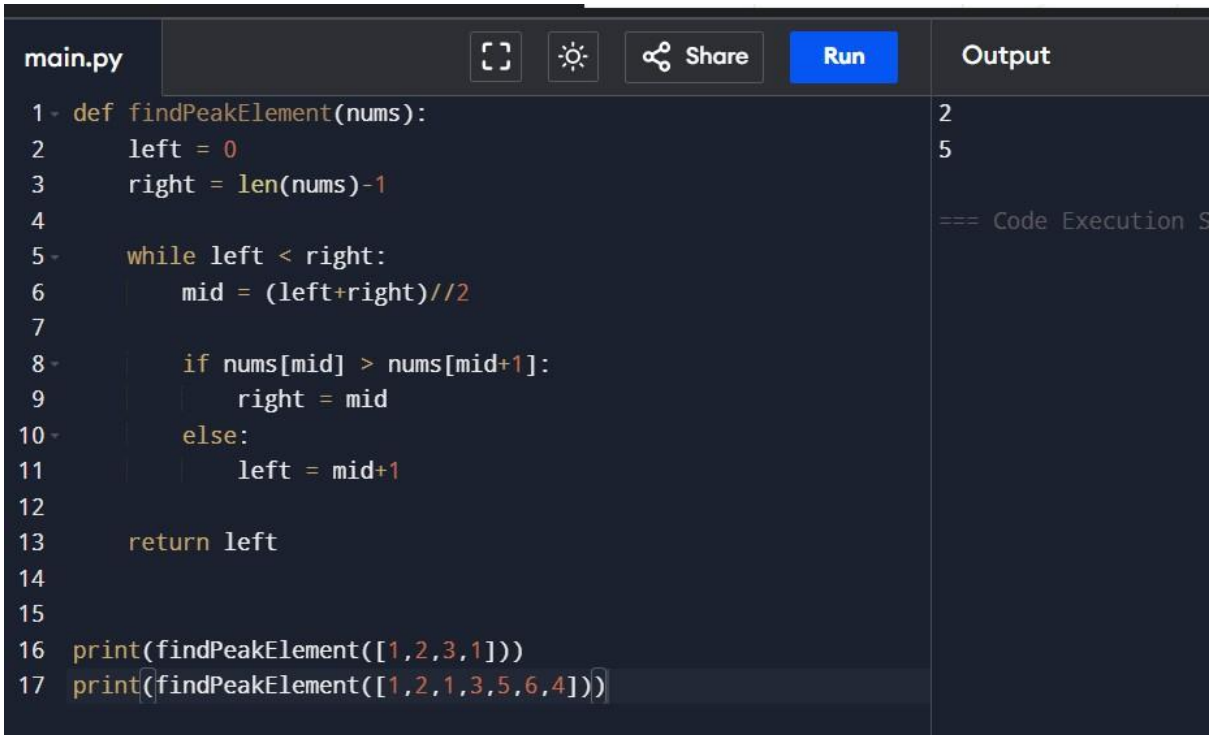
main.py | Share | Run | Output

```
def findKthPositive(arr, k):
    current = 1
    i = 0

    while k > 0:
        if i < len(arr) and arr[i] == current:
            i += 1
        else:
            k -= 1
            if k == 0:
                return current
        current += 1


print(findKthPositive([2,3,4,7,11],5))
print(findKthPositive([1,2,3,4],2))
```

```
9
6

=== Code Execution Successful ===
```

6.A peak element is an element that is strictly greater than its neighbors. Given a 0-indexed integer

array nums, find a peak element, and return its index. If the array contains multiple peaks, return the index to any of the peaks. You may imagine that nums[-1] = nums[n] = -∞. In other words, an element is always considered to be strictly greater than a neighbor that is outside the array. You must write an algorithm that runs in O(log n) time.



7.Given two strings needle and haystack, return the index of the first occurrence of needle in haystack, or -1 if needle is not part of haystack.

8.Given an array of string words, return all strings in words that is a substring of another word. You can return the answer in any order. A substring is a contiguous sequence of characters within a string

Example 1:
Input: words = ["mass","as","hero","superhero"]
Output: ["as","hero"]
Explanation: "as" is substring of "mass" and "hero" is substring of "superhero".
["hero","as"] is also a valid answer.

Example 2:
Input: words = ["leetcode","et","code"]
Output: ["et","code"]
Explanation: "et", "code" are substring of "leetcode".
Example 3:
Input: words = ["blue","green","bu"]
Output: []
Explanation: No string of words is substring of another string.

main.py Share Run Output

```
def stringMatching(words):
    ans=[]

    for i in range(len(words)):
        for j in range(len(words)):
            if i!=j and words[i] in words[j]:
                ans.append(words[i])
                break

    return ans


print(stringMatching(["mass","as","hero","superhero"]))
print(stringMatching(["leetcode","et","code"]))
print(stringMatching(["blue","green","bu"]))
```

```
['as', 'hero']
['et', 'code']
[]

=== Code Executi
```

9.Write a program that finds the closest pair of points in a set of 2D points using the brute force approach.

Input:

- A list or array of points represented by coordinates (x, y).

Points: [(1, 2), (4, 5), (7, 8), (3, 1)]
Output:

- The two points with the minimum distance between them.
- The minimum distance itself.

Closest pair: (1, 2) - (3, 1) Minimum distance: 1.4142135623730951

main.py

```python
import math

def closest_pair(points):

    min_distance=float('inf')
    pair=None

    for i in range(len(points)):
        for j in range(i+1,len(points)):

            d=math.dist(points[i],points[j])

            if d<min_distance:
                min_distance=d
                pair=(points[i],points[j])

    return pair,min_distance


points=[(1,2),(4,5),(7,8),(3,1)]

pair,distance=closest_pair(points)

print("Closest Pair:",pair)
print("Minimum Distance:",distance)
```

Output

```
Closest Pair: ((1, 2), (3, 1))
Minimum Distance: 2.23606797749979

=== Code Execution Successful ===
```

10.Write a program to find the closest pair of points in a given set using the brute force approach. Analyze the time complexity of your implementation. Define a function to calculate the Euclidean distance between two points. Implement a function to find the closest pair of points using the brute force method. Test your program with a sample set of points and verify the correctness of your results. Analyze the time complexity of your implementation. Write a brute-force algorithm to solve the convex hull problem for the following set S of points? P1 (10,0)P2 (11,5)P3 (5, 3)P4 (9, 3.5)P5 (15, 3)P6 (12.5, 7)P7 (6, 6.5)P8 (7.5, 4.5).How do you modify your brute force algorithm to handle multiple points that are lying on the sameline?

Given points: P1 (10,0), P2 (11,5), P3 (5, 3), P4 (9, 3.5), P5 (15, 3), P6 (12.5, 7), P7 (6, 6.5), P8 (7.5, 4.5).

output: P3, P4, P6, P5, P7, P1

main.py

```python
    p=left

    while True:

        hull.append(p)

        q=points[0]

        for r in points:

            if q==p or orientation(p,q,r)<0:
                q=r

        p=q

        if p==left:
            break

    return hull


points=[(10,0),(11,5),(5,3),(9,3.5),(15,3),(12.5,7),(6,6.5
    ),(7.5,4.5)]

print(convexHull(points))
```

Output

```
[(5, 3), (10, 0), (15, 3), (12.5, 7), (6, 6.5)]

=== Code Execution Successful ===
```

11.Write a program that finds the convex hull of a set of 2D points using the brute force approach.

Input:

- A list or array of points represented by coordinates (x, y).

Points: [(1, 1), (4, 6), (8, 1), (0, 0), (3, 3)]

Output:

- The list of points that form the convex hull in counter-clockwise order.

Convex Hull: [(0, 0), (1, 1), (8, 1), (4, 6)]

main.py Share Run Output

```
    hull=[]

    p=left

    while True:

        hull.append(p)

        q=points[0]

        for r in points:

            if q==p or orientation(p,q,r)<0:
                q=r

        p=q

        if p==left:
            break

    return hull


points=[(1,1),(4,6),(8,1),(0,0),(3,3)]

print(convexHull(points))
```

```
[(0, 0), (8, 1), (4, 6)]

=== Code Execution Successful ===
```

12.You are given a list of cities represented by their coordinates. Develop a program that utilizes exhaustive search to solve the TSP. The program should:

1. Define a function distance(city1, city2) to calculate the distance between two cities (e.g., Euclidean distance).
2. Implement a function tsp(cities) that takes a list of cities as input and performs the following:
    - Generate all possible permutations of the cities (excluding the starting city) using itertools.permutations.
    - For each permutation (representing a potential route):
        - Calculate the total distance traveled by iterating through the path and summing the distances between consecutive cities.
        - Keep track of the shortest distance encountered and the corresponding path.
    - Return the minimum distance and the shortest path (including the starting city at the beginning and end).

3. Include test cases with different city configurations to demonstrate the program's functionality. Print the shortest distance and the corresponding path for each test case.

Test Cases:

1. **Simple Case:** Four cities with basic coordinates (e.g., [(1, 2), (4, 5), (7, 1), (3, 6)])
2. **More Complex Case:** Five cities with more intricate coordinates (e.g., [(2, 4), (8, 1), (1, 7), (6, 3), (5, 9)])

Output:

Test Case 1:

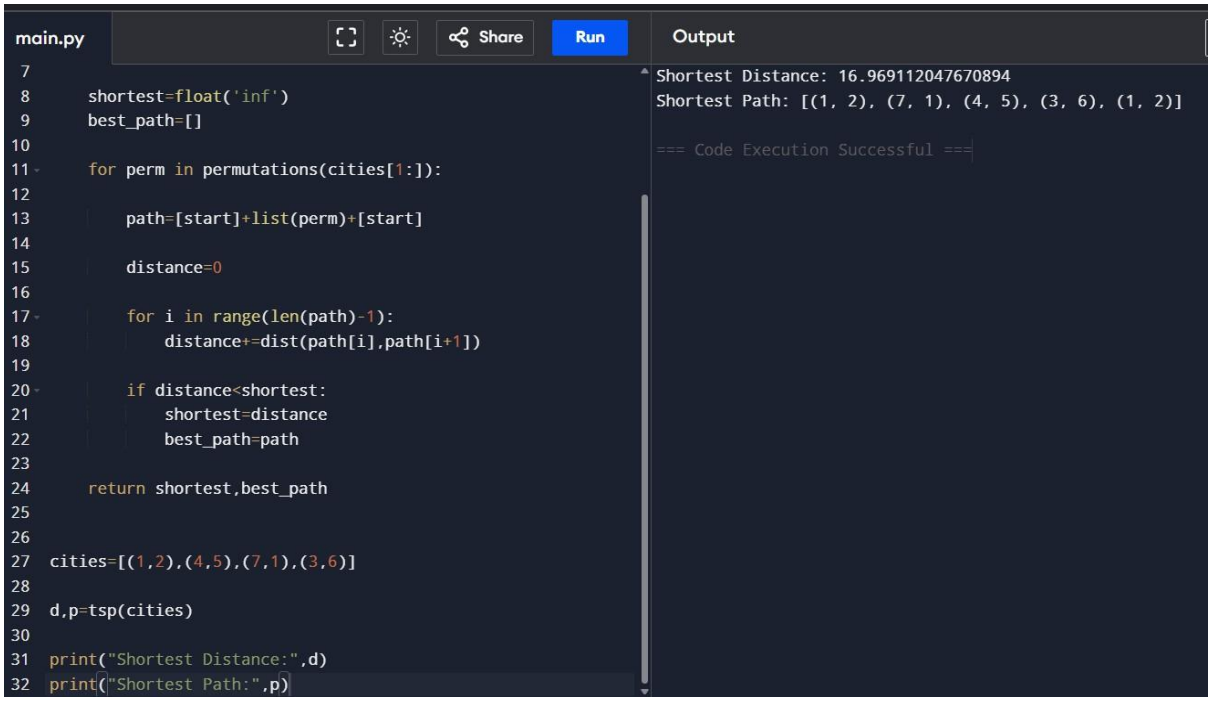
Shortest Distance: 7.0710678118654755

Shortest Path: [(1, 2), (4, 5), (7, 1), (3, 6), (1, 2)]

Test Case 2:

Shortest Distance: 14.142135623730951

Shortest Path: [(2, 4), (1, 7), (6, 3), (5, 9), (8, 1), (2, 4)]



13.You are given a cost matrix where each element cost[i][j] represents the cost of assigning worker i to task j. Develop a program that utilizes exhaustive search to solve the assignment problem. The program should Define a function total_cost(assignment, cost_matrix) that takes an assignment (list representing worker-task pairings) and the cost matrix as input. It iterates through the assignment and calculates the total cost by summing the corresponding costs from the cost matrix Implement a function assignment_problem(cost_matrix) that takes

the cost matrix as input and performs the following Generate all possible permutations of worker indices (excluding repetitions).

Test Cases:

Input

1. **Simple Case:** Cost Matrix:

   [[3, 10, 7],

   [8, 5, 12],

   [4, 6, 9]]

2. **More Complex Case:** Cost Matrix:

   [[15, 9, 4],

   [8, 7, 18],

   [6, 12, 11]]

Output:

Test Case 1:

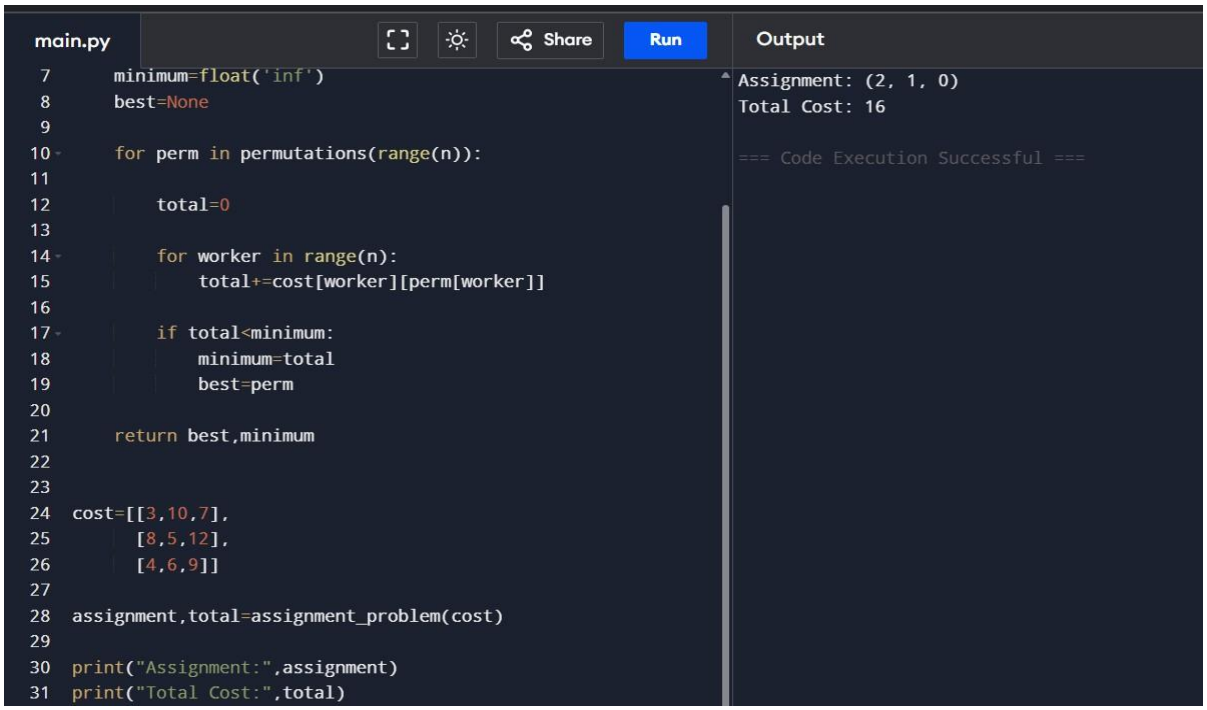
Optimal Assignment: [(worker 1, task 2), (worker 2, task 1), (worker 3, task 3)]

Total Cost: 19

Test Case 2:

Optimal Assignment: [(worker 1, task 3), (worker 2, task 1), (worker 3, task 2)]

Total Cost: 24



14.You are given a list of items with their weights and values. Develop a program that utilizes exhaustive search to solve the 0-1 Knapsack Problem. The program should:

1. Define a function total_value(items, values) that takes a list of selected items (represented by their indices) and the value list as input. It iterates through the selected items and calculates the total value by summing the corresponding values from the value list.
2. Define a function is_feasible(items, weights, capacity) that takes a list of selected items (represented by their indices), the weight list, and the knapsack capacity as input. It checks if the total weight of the selected items exceeds the capacity.

Test Cases:

1. **Simple Case:**

- Items: 3 (represented by indices 0, 1, 2)
- Weights: [2, 3, 1]
- Values: [4, 5, 3]
- Capacity: 4

2. **More Complex Case:**

- Items: 4 (represented by indices 0, 1, 2, 3)
- Weights: [1, 2, 3, 4]
- Values: [2, 4, 6, 3]
- Capacity: 6

Output:

Test Case 1:

Optimal Selection: [0, 2] (Items with indices 0 and 2)

Total Value: 7

Test Case 2:

Optimal Selection: [0, 1, 2] (Items with indices 0, 1, and 2)

Total Value: 10